Hybrid Classical-Quantum Generative Algorithms for Financial Modelling and Prediction

Santanu Ganguly
SandboxAQ
Surrey TW15 1QE, UK
santanug@ieee.org
santanu.gangoly@gmail.com
santanu.ganguly@sandboxaq.com

Xing Liang
Kingston University London
Surrey KT1 2EE, UK
X.Liang@kingston.ac.uk

Dimitrios Makris
Kingston University London
Surrey KT1 2EE, UK
D.Makris@kingston.ac.uk

2025 International Conference on Intelligent Control, Computing and Communications (IC3) | 979-8-3315-2749-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/IC363308.2025.10956507

***Abstract*— Quantum machine learning (QML) is a cross-disciplinary subject made up of two of the most exciting research areas: quantum computing and classical machine learning (ML), with ML and artificial intelligence (AI) being projected as the first fields that will be impacted by the rise of quantum machines. Quantum computers are being used today in drug discovery, material & molecular modelling and finance. In this work, we explore emerging research areas in the application of hybrid classical quantum machine learning (QML) in finance. We focus on hybrid classical-quantum models that have gained significant interest in the financial world for various applications. We utilise a real-world cryptocurrency dataset to train a quantum generative adversarial networks (qGAN), and implement the real-life NVIDIA stock price data on quantum LSTM (QLSTM), comparing its performance with classical Long Short Term Memory (LSTM) model. Additionally, we investigate the application of quantum circuit Born machine (QCBM) using a toy dataset for applications in finance. For the qGAN, we define quantum circuits for generators and showcase the potential for future quantum advantage in finance through QML, possibly in combination with QLSTM and QCBM. Our findings suggest that, while training classical data is more efficient on LSTM, the test predictions of QLSTM is slightly superior to its classical counterpart making it a promising candidate for financial forecasting and analysis for timeseries type data.**

Keywords—Quantum Generative Adversarial Network (qGAN), QLSTM, QCBM, QML

I. Quantum Computing in Finance

Quantum finance is an upcoming cross-discipline of financial engineering with the integration of quantum field theory, classical finance theory, computer science, and artificial intelligence (AI) technology [21]. Quantum machine learning (QML) is a cross-disciplinary subject made up of two of the most exciting research areas: quantum computing and classical machine learning. Recent advances of quantum computing technologies and promise of quantum advantage in QML have led research efforts to consider how to utilize them in industries and a major focus area is aspects of finance [1]. *This work significantly extends previous works done by authors [21, 49] by adding LSTM, QLSTM for time-series, extensive GPU and CUDA-Q based training/test loss analysis and new real-life stock data for the qGAN section.*

Financial institutions are performing enormous tasks of numerical calculation in their daily works, and promise of quantum computing to speed up such tasks is enticing. For example, one of such tasks is pricing of financial derivatives: large banks typically have a huge number of derivatives written on various types of assets such as stock price, foreign exchange rate, interest rate, commodity and so on. Therefore, pricing of derivatives is an important issue for them. Another very important aspect of quantum finance is option pricing, closely related to financial derivatives [2, 49]. Efforts of marrying quantum machine learning to stock market dynamics by leveraging the probabilistic nature of quantum computing and algorithms have been ongoing for forecasting and risk-analysis as well [3].

Quantum computers have been found to be useful in solving several well-known financial engineering problems [4]. Simulation of portfolio loss distribution is achieved in classical computing by using the classical Monte Carlo method by correlating underlying economic factors and associated asset relationships as a way to simulate all possible loss events. This is done by performing iterations over N samples taken from normal distribution of the data prior to averaging several associated trajectories requiring large number of samples. This process tends to make classical Monte Carlo an expensive process, both computationally and time-wise. On the other hand, Quantum computers are deemed to be a natural fit to solve Monte Carlo simulations - which are fundamental for derivatives pricing and related tasks, since they can be viewed as true random number generators. Even though several works have been produced in recent times on theoretical viability of these algorithms, practical implementation of these with real life data are yet to be done.

Quantum generative models, due to their inherent nature, promises to deliver quantum advantages on NISQ devices. This paper focuses on novel implementation of the following QML algorithms as applied to financial industry: a) application of quantum-classical hybrid Quantum Generative Adversarial Networks (qGAN) treated with new data leveraging authors' previous work [49], b) a hybrid classical-quantum Quantum Long Short Memory (QLSTM) for stock prediction using real-life stock price data of NVIDIA and c) Quantum Circuit Born Machine (QCBM) motivated by [5] leveraging authors' previous work [49], showing that Restricted Boltzmann Machines (RBM) outperforms parametric models for this type of financial dataset.

II. Structure of The paper

This paper is structured in the following main parts:

The first part (Section III) briefly describes the fundamentals of classical Generative Adversarial Networks (GAN) and qGAN, associated works done and implementation of qGAN followed by QUBO for optimization for options pricing using real world financial dataset of five cryptocurrencies from Binance exchange. The second part (Section IV) addresses implementation and results of LSTM (Long Short Term Memory) and hybrid classical-quantum QLSTM following the theoretical study of Chen at al. [35] in analysis of real stock market data from NVIDIA during the last 210 days, training and test predictions of the same and finally compares the results drawing conclusions

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

from various metrics. The third part (Section V) addresses the description and implementation of QCBM using a synthetic dataset and discuss the results and future applications of the algorithm. Section VI addresses the methods and results of each of the above approach before we draw conclusions.

III. GAN AND QGAN

GANs (Generative Adversarial Networks) were introduced in 2014 in a well cited paper [6] by Goodfellow et al. and tested initially on image datasets [7, 8], medicine [9, 10], Quantitative Finance [11], for portfolio optimization [12], fraud detection [13], trading model optimization [14] and generation of time series [15, 16]. The advantage of GANs is that they can be trained in an unsupervised way.

A qGAN (quantum GAN) is a partial classical GAN with part of the model implemented via quantum neural networks (QNN). The training process of a qGAN is analogous to that of a classical GAN. qGANs are relatively very new and active research area, especially in financial domain. Research in qGANs are being done in several areas in finance, such as, to generate probability distributions for univariate distributions [22, 23] and for multivariate variate distributions [24, 25].

A. Classical GAN

GAN generates data closely resembling the patterns and properties of a given dataset is by utilizing two neural networks (NN) as main components: a generator, which is a "generative" NN and a discriminator NN. GAN implementations, however, have been found to often exhibit instability issues, vanishing gradient and mode collapse [17]. There have been efforts to improve the efficiency of GAN, for example, Lloyd and Weedbrook (2018) [19] and Dallaire-Demers and Killoran [20] introduced a quantum component to GANs, where the data consists of quantum states or classical data while the two players are equipped with quantum information processors.

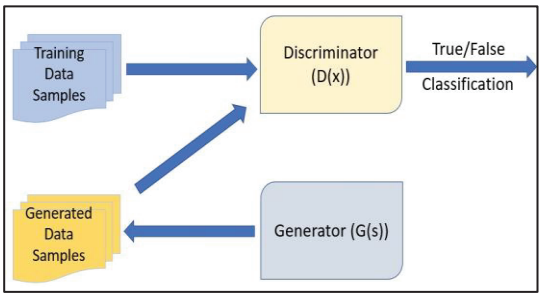

Fig 1. A classical GAN process. The Generator tries to create indistinguishable data samples from the training data. The differentiator tries to discern the generated data from the original data and outputs a True/False binary-classification.

If a classical training dataset $X = \{x_1 \dots x_{s-1}\}$ is sampled from an unknown probability distribution, S is a set of seeds fed to the generator and G_θ and D_φ are the actions of the generator and the discriminator with θ and φ as parameters, then the discriminator training loss is given by

$$L_D = -\frac{1}{|X| + |S|}\left(\sum_{x \in X} \log D_\varphi(x) + \sum_{s \in S} \log\left(1 - D_\varphi\big(G_\theta(s)\big)\right)\right) \quad (1)$$

In equation (1), $|X|$ and $|S|$ denote the respective size of sets. The job of the discriminator is to minimize this loss. The goal of training the generator is to maximise L_D – in order to to deceive the discriminator into classifying the generated data as real data – which would be similar to minimizing $-L_D$. In equation (1), contribution of the first term is constant in the training of the generator as it does not depend on $G_\theta(s)$. Therefore, the goal of the generator training session is theoretically given by the minimization of the loss

$$L_G{}^* = \frac{1}{|S|}\sum_{s \in S} \log\left(1 - D_\varphi\big(G_\theta(s)\big)\right) \quad (2)$$

However, practical considerations [6] prove that the more stable solution for L_G is minimization of the loss function

$$L_G = -\frac{1}{|S|}\sum_{s \in S} \log\left(D_\varphi\big(G_\theta(s)\big)\right) \quad (3)$$

The generator loss function L_G decreases while the generator is trained making it more likely for the generated data to be erroneously classified as real data by the discriminator. It has been shown [6] that the discriminator assigns a value of $\sim 1/2$ to both $G_\theta(s)$ and $D_\varphi(x)$ at its optimal equilibrium since it is unable to distinguish between the real and generated data, giving $L_G = L_D = -\log 1/2 \approx 0.6931$.

B. Quantum GAN (qGAN)

A qGAN is a partial classical GAN with part of the model implemented via quantum neural networks (QNN). The training process of a qGAN is analogous to that of a classical GAN. qGANs are relatively very new and active research area, especially in financial domain. Research in qGANs are being done in several areas in finance, such as, to generate probability distributions for univariate distributions [22, 23] and for multivariate variate distributions [24, 25].

qGAN principles were introduced independently by Lloyd and Weedbrook [19] and also by Dallaire-Demers and Killoran [20]. The classical GAN problem was transformed by introducing density matrices for the qGAN version [19]. If some data is represented by a density matrix ρ – which does not have to be a pure state – and a generator G_q generating an output density matrix μ, the discriminator tries to discern true data from the generated fake one. To do this, the discriminator makes a positive-operator-valued (POV) measurement with outcomes of True (T) or False (F). This gives the probability that measurement of a positive outcome given the true data is:

$$P(T|\rho) = \mathrm{Tr}(T\rho)$$

The probability that the measurement yields a positive answer is given by:

$$P(T|G) = \mathrm{Tr}(T\rho)$$

In this paper we explore the implementation of a qGAN for the uncertainty model on option pricing using real world data from Binance. The approach was first suggested by Zoufal et al. [26]. To address the qGAN scenario, a parametrised quantum generator is trained to work on a given n-qubit input state $|\psi\rangle$ to an n-qubit output state

$$G_{q\theta}|\psi\rangle_n = \sum_{i=0}^{2^n-1} \sqrt{p_\theta^i}\,|i\rangle_n \quad (4)$$

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

where p_θ^i is the output occurrence sampling probabilities of the basis state i. Starting from n-qubits, a quantum generator $G_{q\theta}$ takes the form of a multi-layer quantum neural network. For each layer $l \in \{1 \dots L\}$, a unitary gate $U(\theta)_l$ acts on the n-qubits at the same time with dependence on hyperparameter $(\theta)_l$. In typical NISQ devices, entanglement is used for pairwise controlled gates to avoid high-order qubit gates in low circuit depths enforced by limited number of available qubits. Hence, for each layer l, U is composed of a single or two-qubit gates only. The quantum generator is implemented by a variational form. The general form of each layer gate of the L-layer neural network is given by

$$U(\theta)_l = \left\{ \bigotimes_{j=1}^{n} R_X(\theta_j) Q^{(j \bmod n)+1}(\theta_{j,imp}) \right\}$$

$$\left\{ \left(\bigotimes_{j=1}^{n} R_Z(\theta_{j,Z,l}) \right) \left(\bigotimes_{j=1}^{n} R_X(\theta_{j,X,l}) \right) \left(\bigotimes_{j=1}^{n} R_Z(\theta_{j,Y,l}) \right) \right\}$$

where, Q^j indicates that qubit j is the control qubit and the gate action is on qubit $(j+1)$ and $((j \bmod n)+1) = 1+j$ when $j \in \{1 \dots, n-1\}$ and is 1 when $j = n$. The total number of hyperparameters are given by $5nl$, with $5n$ per layer. The discriminator itself, generally may or may not be quantum in nature depending on the nature of the problem. In our particular case, the discriminator is a classical binary classifier. Once the data is trained by qGAN, optimization was done via QUBO (Quadratic Unconstrained Binary Optimization) to predict the best possible crypto currency available.

QUBO unifies a variety of combinatorial optimization problems with a wide range of applications in finance and economics to machine learning that leverage applied optimization models in quantum computing domain [37]. The significance of the QUBO model lies in its capacity to encompass several models in combinatorial optimization and it has been shown by Lucas [37] that the QUBO model is equivalent to the Ising model [38]. The Ising model is a non-physical, mathematical model of ferromagnetism in reference to statistical mechanics that uses discrete variables which represent the magnetic dipole moments of spin states, (either +1 or -1). The data, which are just the spin states, are organized as a lattice so that each spin can interact with its neighbors. QUBO has emerged as a powerful tool for solving computationally hard combinatorial optimization problems that are ubiquitous in many important applications like large-scale portfolio management [42, 38].

IV. LSTM AND QLSTM

A. LSTM

LSTM (Long Short-Term Memory) [36] is a type of recurrent Neural Network (RNN) applicable to a broad range of problems aiming to analyze or classify sequential data. A *recurrent network* layer has the basic ability to process sequential input data $x_1, x_2, \cdots, x_n$ by updating its hidden state h_t one time-step at a time as element of the sequence x_t passes through it. The hidden state constitutes a vector which is comprised of units. The length of the vector equals the number of units in the cell, where the cell is supposed to have full knowledge of the sequence. In comparison to the basic RNN with one state vector h_i, LSTM structure adds a new state vector c_i and introduces a gate control mechanism for forgetting and updating of information through the gate control unit as shown in Fig. 2 below [35].

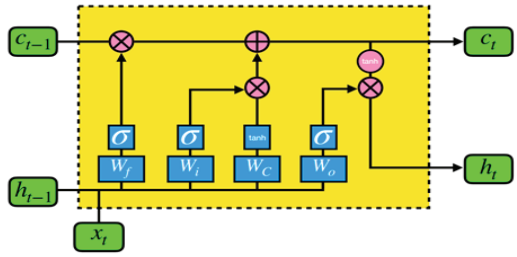

Fig 2. A classical LSTM, from Chen et al. [35]

In Fig. 2, σ denotes the sigmoid function, $\{W_n\}$ are classical neural networks $(n = f, i, C, o)$, where f represents the forget block, i represents the input block, C represents a new state cell candidate, and o represents the output block. LSTM separates the internal memory and output into two variables using three main types of gates: input gate, forget gate and output gate. An LSTM cell allows the gradients to flow unchanged and can be interpreted as the memory of the LSTM cell with two memory components c_t and h_t, as opposed to RNN, which has only one memory component h_i. Since LSTM enhances the memory capabilities of an RNN, they are numerically more stable and capable of delivering more efficient predictions for temporal data such as time series. RNN has been investigated for applications in learning quantum evolution dynamics from experimental data with sequential characteristic [43,44]. Chen et al., in [35], successfully showed that the classical LSTM can be extended to hybrid classical-quantum realm by replacing the classical definition of $h_t = o_t * \tanh(c_t)$ with $h_t = VQC_5(o_t * \tanh(c_t))$. In other words, they defined the fundamental unit of the QLSTM architecture, a QLSTM cell, as a stacked together VQC (variational quantum circuit) blocks, shown in Fig 3. The σ and tanh blocks represent the sigmoid and the hyperbolic tangent activation function, respectively, x_t is the input at time t, h_t is the hidden state, c_t is for the cell state, and y_t is the output. In essence, the QLSTM model replaces the classical LSTM cells with VQCs.

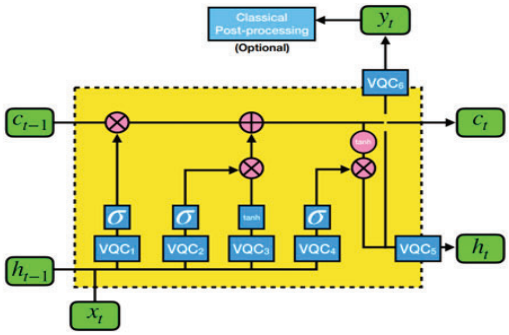

Fig 3. A QLSTM, from Chen et al. [35]

The basic idea behind a VQC is to run a short sequence of gates where some gates are parametrized. The process then reads out the result, makes adjustments to the parameters on a classical computer, and repeats the calculation with the new parameters on the quantum hardware. This way an iterative loop is created between the quantum and the classical processing units, creating classical-quantum hybrid algorithms, called VQA (variational quantum algorithm) [21].

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

Previous research [Chen 46-48] have identified VQAs as algorithms that are efficient against noise in NIST devices. In our case, the VQCs were defined in the same fashion as [35]:

1) Data Encoding Layer: A layer consisting of Hadamard (H) gates, and rotational R_y and R_z gates.

2) A variational layer as described above.

3) A quantum measurement layer.

Classical data fed into a quantum circuit needs to be processed and encoded prior to the operation. The input classical data must be represented as a quantum state in order to be understood by and manipulated on a quantum computer. If we consider classical input data consisting of P datapoints, with R features each, then, for a dataset D,

$$D = x^1, x^2, \dots, x^p, \dots, x^P$$

where, x^p is a R-dimensional vector and $p = 1,2, \dots, P$. This data can be embedded or encoded into n quantum *subsystems*. These subsystems can be n-qubits for discrete-variable (DV) or n-qumodes for continuous-variable (CV) quantum computing. In order to prepare input for a quantum algorithm as a quantum state, a quantum circuit is defined which prepares the corresponding state. This circuit can be generated in a classical preprocessing step as shown in Fig 4.

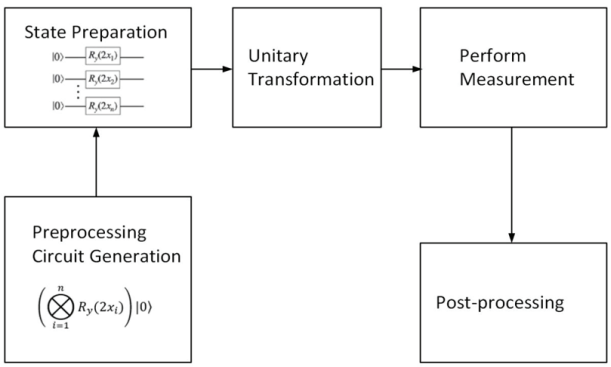

Fig 4. Processing classical inputs for quantum state [21]

In Fig. 4, the process is as follows:
Let $\mathrm{x} = \mathrm{x}_1, \mathrm{x}_2, \dots, \mathrm{x}_\mathrm{n} \in \mathrm{R}^\mathrm{n}$. The tensor product encoding on each component x_i is given by

$$|\mathrm{x}_\mathrm{i}\rangle = \cos\cos \mathrm{x}_\mathrm{i}\, |0\rangle + \sin\sin \mathrm{x}_\mathrm{i}\, |1\rangle$$

which gives the following complete vector representation

$$|x\rangle = \left(\frac{cos\ cos\ x_1}{sin\ sin\ x_1}\right) \otimes \left(\frac{cos\ cos\ x_2}{sin\ sin\ x_2}\right) \otimes \dots \otimes \left(\frac{cos\ cos\ x_n}{sin\ sin\ x_n}\right)$$

The Pauli Y and Z matrices when exponentiated, give rise to the rotation operators, which rotate the Bloch about the x, y, and z axes, by a given angle θ:

$$R_x(\theta) \equiv e^{-i\frac{\theta}{2}X} = cos\ cos\ \frac{\theta}{2} I - i\ sin\ sin\ \frac{\theta}{2} X = \left[cos\ cos\ \frac{\theta}{2} - i\ sin\ sin\ \frac{\theta}{2} - i\ sin\ sin\ \frac{\theta}{2}\ \ cos\ cos\ \frac{\theta}{2}\right]$$

$$R_y(\theta) \equiv e^{-i\frac{\theta}{2}Y} = cos\ cos\ \frac{\theta}{2} I - i\ sin\ sin\ \frac{\theta}{2} Y = \left[cos\ cos\ \frac{\theta}{2}\ \ -sin\ sin\ \frac{\theta}{2}\ \ sin\ sin\ \frac{\theta}{2}\ \ cos\ cos\ \frac{\theta}{2}\right]$$

$$R_Z(\theta) \equiv e^{-i\frac{\theta}{2}Z} = cos\ cos\ \frac{\theta}{2} I - i\ sin\ sin\ \frac{\theta}{2} Z = \left[e^{-i\frac{\theta}{2}}\ 0\ 0\ e^{i\frac{\theta}{2}}\right]$$

Hence, applying the unitary $R_y(2x)$ to the vector $|x\rangle$ in Fig. 5, we have,

$$R_y(2x) = [cos\ cos\ x\ \ -sin\ sin\ x\ \ sin\ sin\ x\ \ cos\ cos\ x\ ]$$

which implies that

$$R_y(2x_i)|0\rangle = cos\ cos\ x_i\ |0\rangle + sin\ sin\ x_i\ |1\rangle = x_i$$

The above equation, in turn, implies that,

$$\left(\otimes_{i=1}^{n} R_y(2x_i)\right)|0\rangle = \left(\frac{cos\ cos\ x_1}{sin\ sin\ x_1}\right) \otimes \left(\frac{cos\ cos\ x_2}{sin\ sin\ x_2}\right) \otimes \dots \otimes \left(\frac{cos\ cos\ x_n}{sin\ sin\ x_n}\right) = |x\rangle \qquad (5)$$

where we obtain the original vector $|x\rangle$ back. In equation 5, $\otimes_{i=1}^{n} R_y(2x_i)$ is the tensor product between all of $R_y(2x_i)$ and state $|0\rangle$ across the entire range of i. The preprocessing step can generate the circuit of Fig. 4 such that the state preparation step will prepare the quantum representation of x. The circuit can then be generated as a code, prepended to the algorithm proper, send to a quantum computer or simulator, which is then executed there.

Chen at al. [35] showed various exploratory use cases of QLSTM. This work is a proof of concept that QLSTM can be implemented in a novel way to enhance stock price prediction, offering comparable and arguably better results than its classical counterpart. We implement both LSTM and QLSTM to predict the stock prices of NVIDIA for 210 days till August 2024, with the same number of features.

V. Quantum Circuits Born Machine (QCBM)

QCBMs are probably the most well-known example of quantum generative models promising to exhibit quantum advantage on NISQ devices. Originally, reliable synthetic market modelling was done through sampling from parametric models that can be easily calibrated. Examples are multivariate normal distribution of risk factor equities or a Gaussian copula. However, there are well known issues with these approaches [33]. QCBMs were introduced by [27] and consists of several layers of adjustable and fixed gates with measurement operators. They sample from the fundamentally quantum distribution underlying a pure state of a quantum system. QCBMs are a specific form of Born machines [28] which are fundamentally quantum models that achieves synthetic data generation (SDG) by generating samples according to Born's rule. These non-classical models have been proven to in at least its expressive power [29]. This expressive power allows it to represent certain distributions efficiently which cannot be done by any classical model, such as, those used in a demonstration of quantum computational

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

supremacy [30]. Generally, a binary sample $x \in \{0,1\}^n$ is generated from a quantum state $\emptyset$:

$$\boldsymbol{x} \sim p(\boldsymbol{x}) = \mathrm{Tr}(|\boldsymbol{x}\rangle\langle\mathbf{x}|\emptyset)$$

where $|\boldsymbol{x}\rangle\langle\mathbf{x}|$ is the projection on the computational basis state denoted by $\boldsymbol{x}$. A parametrization of the state $\emptyset \to \emptyset_\theta$ is performed to obtain a trainable machine learning model. If the parametrization state is a pure state then, $\emptyset_\theta = |\psi_\theta\rangle\langle\psi_\theta|$ and the parametrized distribution is given by

$$\boldsymbol{x} \sim p_{\theta,qcbm}(\boldsymbol{x}) = |\langle\boldsymbol{x}|\psi_\theta\rangle|^2 \quad (6)$$

If the state is generated by a quantum circuit (and not, for example by a continuous time Hamiltonian evolution), then the model is referred to as QCBM [27]. In this form, the parametrized quantum state prepared by a quantum circuit can be measured once the model is trained and the measurement results provide an approximate sample from the distribution. Additionally, using quantum randomness as a sample generation mechanism by default improves the requirement to input randomness into the model as is done in case of GANs. A generalization of this method can be achieved by considering mixed states and not just pure states, and that in turn results in quantum Hamiltonian based models [31].

QCBM, is a parametrized quantum circuit with inputs initialized as $|0\rangle$ in the computational basis, where a layer of adjustable one-qubit gates is followed by a layer of fixed two-qubit gates. The final layer consists of measurement operators producing a bitstring sample from the learned distribution. Hence, in order to build a QCBM, one needs to specify the number of layers, type of fixed gates for each layer and type of adjustable gates. The expectation of experimental proof of quantum advantage is motivated by the fact that QCBMs have a larger expressive power than classical Random Boltzmann Machines (RBM) in cases when only a polynomial number of parameters are allowed and generation of an independent sample from a learned distribution can be done in a single run of the quantum circuit of the QCBM which, when compared to the $\sim 10^4$ backwards and forwards runs in the case of the RBM to reach a state of thermal equilibrium [32]. QCBM has a bitstring as an output that represents a sample from the probability distribution encoded in the quantum state and resolves many of the challenges of the original method. The circuit is designed to transform an initial state $|0\rangle^{\otimes n}$ into a final state from which a sample is generated by means of measuring the qubits on the computational basis.

VI. Methods and Results

This section addresses the implementation methods and discussions about the results. All coding and simulations for QCBM and qGAN were done on NVIDIA RTX 3070 GPU on Linux Ubuntu 22.04 LTS leveraging CUDA-Q[47]. The LSTM and QLSTM codes were run on Apple 30-core GPU on an M3 MacBook, in addition to the above NVIDIA powered platforms. Libraries used were Qiskit [45] and PennyLane [46]. Lightning.gpu [48] was used to boost QML performance of PennyLane.

A. qGAN for Cryptocurrency Forecast

The qGAN implementation follows the authors' previous work [49] and made up of a quantum circuit that will be able to prepare a 1-qubit state $|\psi_1\rangle$, also called the "true" state, that we want the qGAN to learn. This circuit runs on the first qubit of the model. $|\psi_1\rangle$ acts as the quantum training data for the training circuit. The circuit tries to produce as many $|\psi_1\rangle$ states as possible for the training process. The quantum generator, implemented via a variational form with trainable parameters, runs on the first qubit of the device in order to prepare a state similar to $|\psi_1\rangle$ on the first qubit. The discriminator tries to distinguish between the two states.

Data: Real life data were used via Binance API call for five different crypto currencies with more than 5000 samples: BNBBTC, ETHBTC, LTCBTC, NEOBTC, QTUMETH. Data samples smaller than 5% percentile and bigger than 95% percentile were discarded to reduce the number of qubits required for a reasonable representation of the distribution. The optimization scheme used data batches of 1,000 and was run for 2000 training epochs mainly utilizing IBM's Qiskit 0.39 [39] and PyTorch [40]. For the qGAN implementation, a parametrized quantum circuit $G_{q\theta}$ was created as the quantum generator. The discriminator was created with a classical neural network approach with its output designed to ideally identify fake from real data as a binary classification. An Adam optimizer was used for the gradient based optimization process. Fig. AI in Appendix shows the generator circuit.

First the training data X was loaded. The trained quantum generator was used to load a quantum state which is an approximate model of the target distribution and its goal is to correspond to the n -qubit quantum state $G_{q\theta}|\psi\rangle_n = \sum_{i=0}^{2^n-1}\sqrt{p_\theta^i}|i\rangle_n$. Samples were mapped from the multivariate normal distribution to discrete values. $2^5 = 32$ discrete values were used and all training samples fell under one of these discrete values. Then, histograms in Fig. 6 were generated for each random variable the number of bins were increased and to make the actual discretization visible. PyTorch 2.0 was used for conversion of data arrays into tensors and loading data for training. Then a quantum instance was created for training where the batch size defines the number of shots using Qiskit Aer [39]. In our case, the generator loss functions are parametrized by θ those of the discriminator are defined by φ . The discriminator, which has to differentiate and classify two data points $|\psi_1\rangle$, labeled '1' and the generated state $|\psi_g\rangle$ with a label '0', has a loss function for the quantum format given by,

$$L_{q\varphi} = -0.5[\log(1 - D|\psi_g\rangle) + \log(D|\psi_1\rangle)]$$

The generator loss function is given by

$$L_{G\theta} = -\log(D|\psi_g\rangle)$$

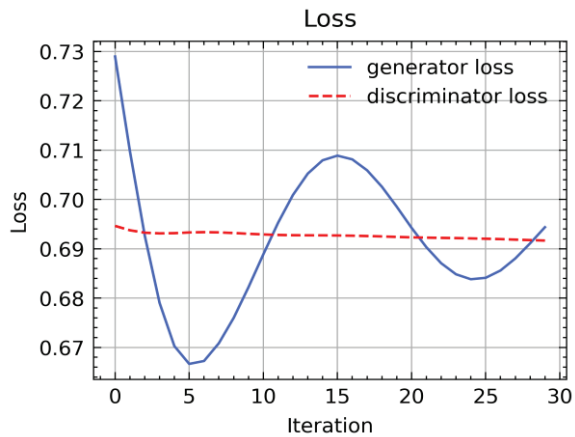

Fig 5. Quantum generator vs. discriminator loss for qGAN

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

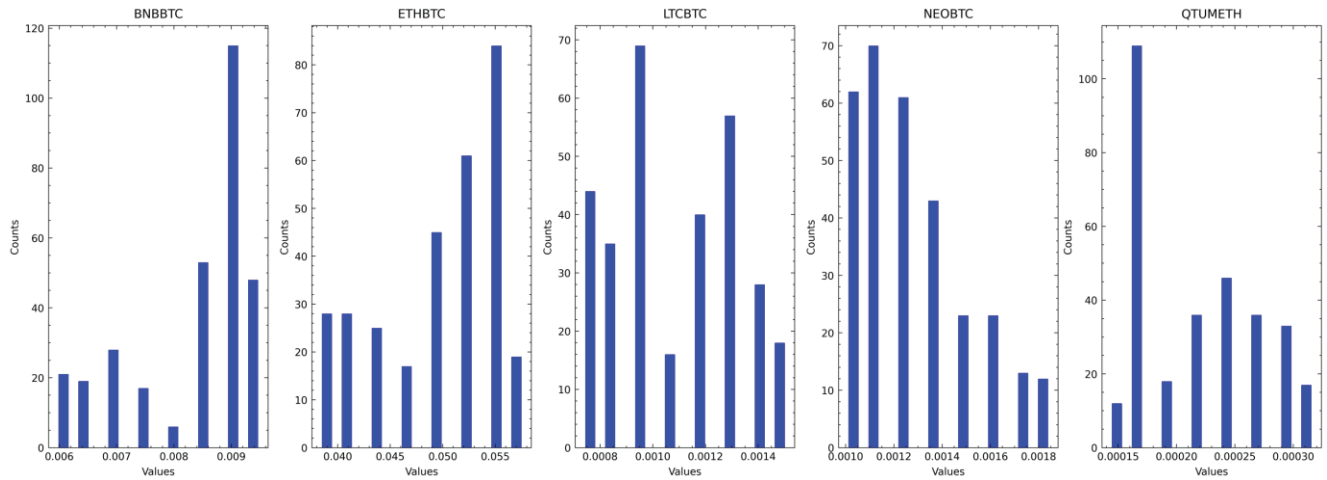

Fig 6. Discrete values and training samples for qGAN. BNBBTC shows a higher probability of being a better buy for recent (15th Dec 2024) data.

The generator vs. discriminator loss functions for the option pricing uncertainty of the Binance data is given in Fig. 5. As shown, the generator not only takes into consideration the singular evolution of an option, but also the correlated evolution of other market assets. As in the classical case, the discriminator assigns a value of $\sim 1/2$ to both $G_\theta(s)$ and $D_\varphi(x)$ at its *optimal equilibrium* where the two loss functions intersect since it is unable to distinguish between the real data and the synthetic data, showing a steadily increasing rise in value with an increase in iterations.

Once the data had been trained via qGAN, `pyqubo` library was used to implement QUBO along with the `CLARABEL` solver from the cvxpy library to select a crypto asset(s) predicted to give maximum return, which in this case was BNBBTC, as is indicated by the histogram distributions in Fig. 6.

B. LSTM & QLSTM for Analysis of NVIDIA Stock Price

A hybrid quantum-classical model was implemented, inspired by Chen et al. [35], for stock price prediction using QLSTM. Stock price prediction is often a challenging task due to the non-linear and volatile nature of the stock market. The proposed model utilises variational quantum circuits (VQCs), used to enhance the classical LSTM's ability to process the non-linear relationships in the stock price data. The VQCs are trained for 50 epochs using the Adam optimizer. The proposed model is evaluated on the NVIDIA dataset spanning 210 days till August 28th, 2024, and the results show that the hybrid quantum-classical model shows results more accurate than the classical LSTM model in terms of prediction accuracy. The classical LSTM model performs better in terms of training time, a result that was not unexpected as the QLSTM model required an additional data encoding step for classical to quantum state as described in Section IV. The classical LSTM architecture is implemented

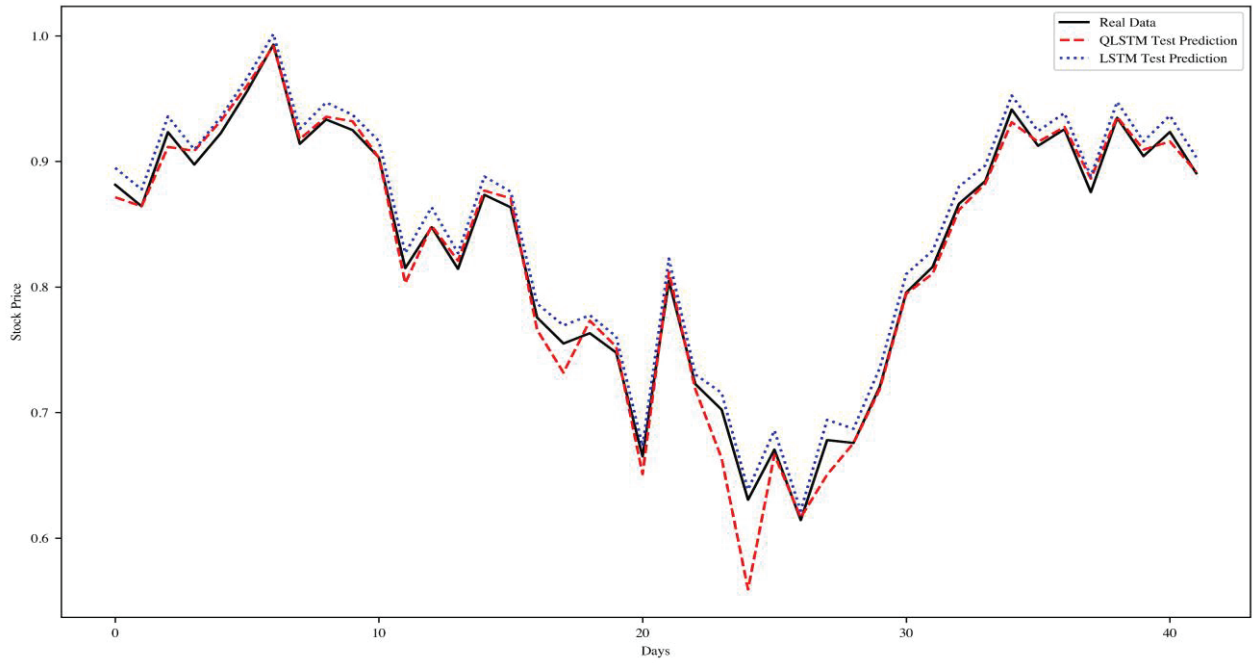

Fig 7. The comparison of Stock prediction for the Test data between the real life data (black), QLSTM (red dash) and classical LSTM (blue dots)

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

using PyTorch [40] with the hidden size 7, parameters size 288 and a linear layer to convert the output to a single target value. For the QLSTM, 6 VQCs (Fig 3) were implemented, in each of which there were 4 qubits giving number of parameters in the QLSTM as 292. Finally, we used PennyLane [54, 58] and Qiskit for training and the simulation of quantum circuits respectively.

We found that both the QLSTM and LSTM successfully train the data and the predictions are closer to each other albeit the QLSTM test predictions, especially around the volatile peaks, as shown in Fig.7, is closer to the real life data than classical LSTM. Notably, the QLSTM learns more information after the first ~5 training epochs than the LSTM, also predicts turning points in the stock market slightly earlier than the LSTM, which could potentially provide an advantage in foreseeing market movements or trends. Moreover, the QLSTM's loss (Fig.8) shows two spikes in comparison to LSTM's and that is relevant to the noise and volatility of the nature of the real-life stock data. Fig. 9 shows the test loss comparison.

Fig 9 sheds more light on why the stock prediction for the *test data* is closer to real life predictions for the QLSTM. The loss spikes start to smooth out with time in the case of QLSTM, as opposed to classical LSTM, where the loss spikes appear pronounced towards the end of the training (Table II). The *compared training plots* are shown in Fig. A2 of the Appendix.

GPU dependence: It was found that the training time per epoch for QLSTM varies significantly depending on the type of GPU used. QLSTM involves simulation of the quantum model, the quantum encoding of the classical data and subsequent training of the same. This is shown in Tables I and II. The training times become significantly efficient with improved GPU models and it was reasoned that a platform more advanced and targeted at quantum simulation such as NVIDIA's CUDA-Q supported DGX Quantum hardware stack should give far superior results, pushing the boundaries of quantum advantage. As such, the training modules were ran on a containerized cuQuantum from NVIDIA [47] and *significant differences* were noted in the performance of the RTX 30XX GPUs on CUDA-Q, as shown in Table III.

TABLE I. TRAINING TIME BASED ON GPU

GPU Model	Training Time (sec.) (per epoch)	
	QLSTM	LSTM
RTX 3060	1600.0	18.0
RTX 3070	780.0	12.0
Apple 30-core GPU M3	22.0	0.05

TABLE II. TRAINING & TEST LOSSES

	Loss	
	Training Loss	Testing Loss
QLSTM	7.2×10^{-3}	5.5×10^{-3}
LSTM	4.5×10^{-3}	6.8×10^{-3}

TABLE III. TRAINING TIME WITH CUDA-Q

GPU Model	Training Time (sec.) (per epoch)	
	QLSTM	LSTM
RTX 3060 with CUDA-Q	500.0	0.5
RTX 3070 with CUDA-Q	18.0	0.04
Apple 30-core GPU M3	22.0	0.05

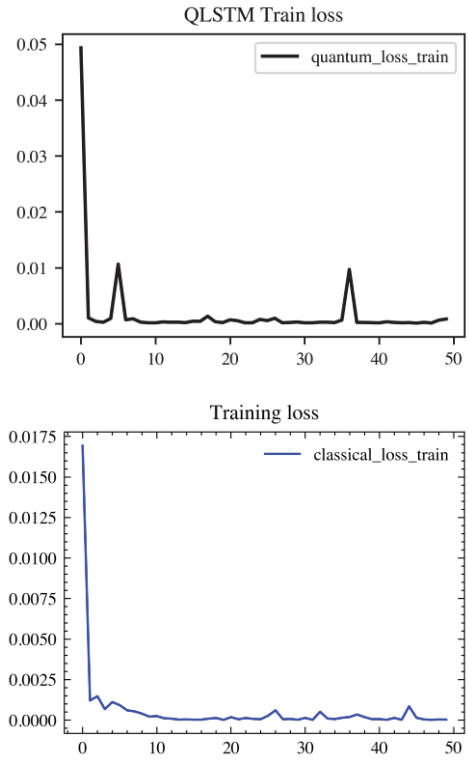

Fig 8. QLSTM and Classical LSTM training loss for stock data

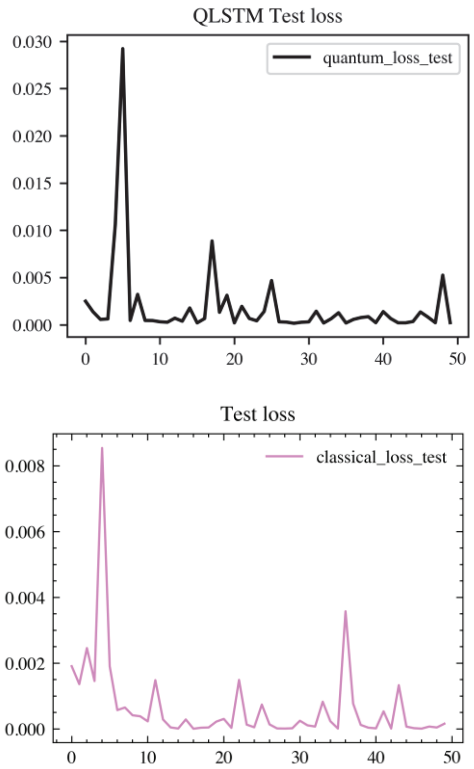

Fig 9. QLSTM and Classical LSTM test loss. NVIDIA stock data

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

Statistical significance testing using a paired t-test on RMSE values was also conducted to assess whether differences in LSTM and QLSTM errors were statistically significant. Model loss curves, prediction plots, and other visualizations were generated to provide qualitative evaluation as shown in Table III. The LSTM performs marginally better for the 50 epochs.

TABLE IV. TRAINING & TEST RMSE

	RMSE values	
	Training Loss	**Testing Loss**
QLSTM	19.05×10^{-3}	15.09×10^{-3}
LSTM	7.5×10^{-3}	12.8×10^{-3}

C. *QCBM for BAS Toy Dataset Analysis*

In machine learning a probability distribution is obtained from a finite set of samples. If the training process is successful, the learned distribution P(x) has sufficient similarity to the actual distribution of the data that it can make correct predictions about unknown situations.

In order to build a QCBM, it is important to specify the number of quantum layers, type of fixed gates for each layer and type of adjustable gates. Initially a histogram (Fig. 10) was generated to verify that the state vector satisfies the characteristics of equation (6). The QCBM design requires a Variational Quantum Circuit (VQC) or ansatz that has multiple layers. This allows the computation to be repeated as many times as required in order to obtain the desired Probability distribution.

Fig.12 shows the QCBM circuit decomposition as generated by Qiskit. Circuit decomposition of the QCBM was done to verify that the same circuit is repeated in layer 1 and layer 3, using the odd layer, and the second circuit is the even layer. For the dataset, a bars and stripes (BAS) dataset was synthetically generated after Benedetti et al. [27] which was composed of binary images of size 2×2. The qubits were mapped considering the fact that they only take two different values, namely, black-white, 0-255, 0-1. The Basis Encoding method was used for quantum encoding, to convert images into quantum state, where each pixel is represented as a qubit. One of the goal for the data was to obtain an approximation to

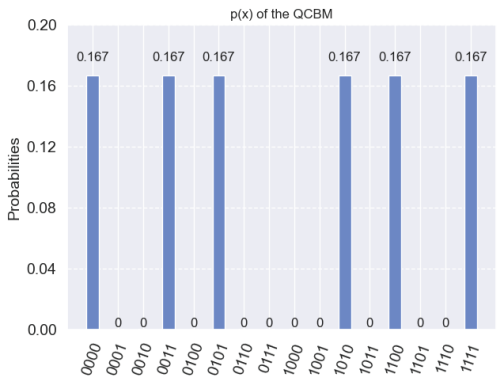

Fig 10. Histogram for Sate Vectors.

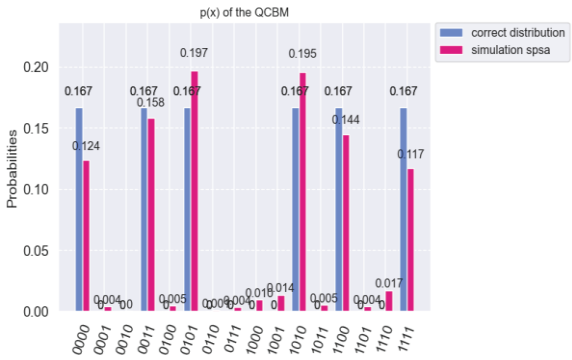

Fig 11. Comparison of real and simulated distribution with SPSA.

the target probability distribution `px_output` or $P(x)$. This was achieved with a quantum circuit parametrized by θ. The layers were defined as 5. Minimization of cost function $\mathbb{C}(\boldsymbol{\theta}) = \frac{1}{-d}\sum_{d=1}^{5} \ln(P(\mathbf{x}^{\boldsymbol{d}})$, where 'd' is the number of layers (5 in this case) and $P(\mathbf{x}) = |\langle \mathbf{x}|\psi(\boldsymbol{\theta})\rangle|^2$ is the probability. A simple variant of $\mathbb{C}(\boldsymbol{\theta})$ was used $\mathbb{C}(\boldsymbol{\theta}) = -\frac{1}{d}\sum_{d=1}^{5} \ln(\max(\epsilon, P_\theta(\mathbf{x}^{\boldsymbol{d}}))$, where $\epsilon > 0$ is a small parametric perturbation. A method called `born_machine(params)` was defined for this purpose. A comparison of the optimizers were done between Cobyla, SPSA and Nelder-Mead. SPSA was found to be the better performing. Finally, in Fig 11, a comparison of the correct probability distribution and simulation by SPSA was done and SPSA simulation appeared to be close to the real distribution.

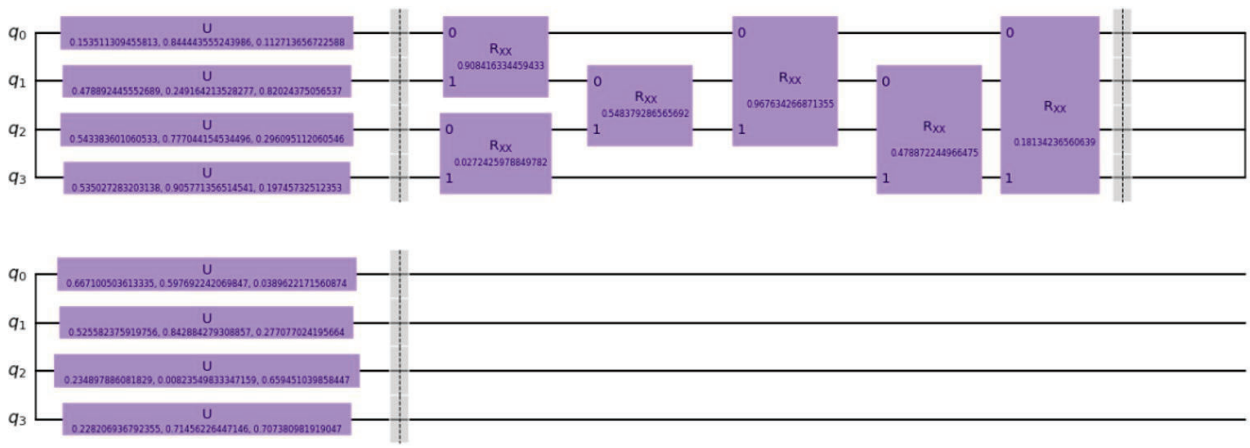

Fig 12. Quantum circuit for QCBM with Qiskit

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

Comparison of the real data with Cobyla had wider gaps with the real `fdata` even though Cobyla, as an optimizer, gave better results.

VII. Conclusion

A study of implementations of qGAN, QLSTM and LSTM, and QCBM were done as a proof-of-concept application of financial data using Qiskit [39] simulator from IBM, PennyLane from Xanadu [41], PyTorch [40] and CUDA-Q libraries from NVIDIA.

qGAN: Real life crypotcurrency data for 5 different cryptos was fed to a VQC driven quantum GAN with a classical discriminator. In the qGAN case, the loss functions of the discriminator and generator behaved as expected. The qGAN produced expected results in far less iterations than it would take a classical GAN. qGAN is a new and exciting area of research that promises robust advantage in treating and modelling financial data. The data treated via the qGAN algorithm was optimized to give a prediction for the best cryptocurrency stock to invest in, which in this timeline, was BNBBTC. Libraries used were Qiskit, PennyLane for quantum machine learning modules and PyTorch for a general machine learning framework.

QLSTM and LSTM: Real life NVIDIA data for 210 days were treated in a time series manner via LSTM and QLSTM implementations. Separate Python files were written for the two processes. Training for LSTM was found be more efficient than for QLSTM. QLSTM training time was found directly dependent on the type of GPU used. However, the test predictions for QLSTM for the NVIDIA data was closer to real data than the test predictions for LSTM. This indicates promises in the direction of applications of classical-quantum hybrid algorithms in financial industry, especially stock prediction from time series. This promises a robust financial model with more training time, fine tuning and combination with qGAN and QCBM. Whereas combination of annealing with qGAN has been attempted by Zhu et al. [34], the similarities between the basic properties of QLSTM and qGAN promises a tighter integration and better efficiency. More research in this area is envisaged. The release of DGX Quantum stack with CUDA-Q by NVIDIA (https://www.nvidia.com/en-gb/solutions/quantum-computing/) promises dynamic research results in this domain in near future as is indicated by the speed-up in quantum simulation performance by the NVIDIA RTX 30XX GPUs on top of cuQuantum (Table III).

The QCBM implementation with the synthetic BAS data produced results with SPSA optimization which promises that further integration with qGAN and QLSTM, can produce further benefits in terms of training and test prediction. This shows that all three of the classical-quantum hybrid algorithms of qGAN, QLSTM and QCBM promise certain advantage from quantum machine learning perspective in financial industry.

qGAN and predictive part of QLSTM showed promises of benefits from quantum domain in comparison to their classical counter parts as run on simulators. The toy dataset for QCBM shows clear indication of optimizing certain data type utilizing SPSA. In recent times promising quantum advantage have been reported by combining QCBM with annealing [34]. Research findings from this paper indicate that QLSTM could be a promising candidate for financial forecasting and analysis, highlighting the potential of hybrid classical-quantum models to offer new insights and improve decision-making in the financial industry. Further work is required in this exciting area and as per recent works, their combination promises deep benefits, especially in cases of temporal data. Further research is intended in areas where qGAN, QLSTM and QCBM can be combined to give promising results.

Acknowledgment

Santanu Ganguly would like to thank SandboxAQ for sponsoring the research and acknowledge the valuable discussions with Tom Lubowe, Product Manager, Quantum Computing, NVIDIA about CUDA-Q.

References

[1] Orús, R., Mugel, S., & Lizaso, E. (2019). Quantum computing for finance: Overview and prospects. Reviews in Physics, 4, 100028.

[2] Stamatopoulos, N., Egger, D.J., Sun, Y., Zoufal, C., Iten, R., Shen, N., & Woerner, S. (2019). Option Pricing using Quantum Computers. Quantum, 4, 291.

[3] Liu, G., & Ma, W. (2022). A quantum artificial neural network for stock closing price prediction. Inf. Sci., 598, 75-85.K. Elissa, "Title of paper if known," unpublished.

[4] Egger, D.J., García Gutiérrez, R., Mestre, J.C., & Woerner, S. (2019). Credit Risk Analysis Using Quantum Computers. IEEE Transactions on Computers, 70, 2136-2145.

[5] Kondratyev, A., & Schwarz, C. (2019). The Market Generator. Econometrics: Econometric & Statistical Methods - Special Topics eJournal.

[6] Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A.C., & Bengio, Y. (2014). Generative Adversarial Nets. NIPS.

[7] Yu, X., Dai, P., Li, W., Ma, L., Shen, J., Li, J., & Qi, X. (2022). Towards Efficient and Scale-Robust Ultra-High-Definition Image Demoireing. ArXiv, abs/2207.09935.

[8] Schawinski, K., Zhang, C., Zhang, H., Fowler, L., & Santhanam, G.K. (2017). Generative Adversarial Networks recover features in astrophysical images of galaxies beyond the deconvolution limit. ArXiv, abs/1702.00403.

[9] Anand, N., & Huang, P. (2018). Generative modeling for protein structures. Neural Information Processing Systems.

[10] Zhavoronkov, A. *et al* (2019). Deep learning enables rapid identification of potent DDR1 kinase inhibitors. Nature Biotechnology, 37, 1038 - 1040.

[11] Ruf, J., & Wang, W. (2020). Hedging With Linear Regressions and Neural Networks. Journal of Business & Economic Statistics, 40, 1442 - 1454.

[12] Lu, J., & Yi, S. (2022). Autoencoding Conditional GAN for Portfolio Allocation Diversification. ArXiv, abs/2207.05701.

[13] Assefa, S.A. (2020). Generating synthetic data in finance: opportunities, challenges and pitfalls. Proceedings of the First ACM International Conference on AI in Finance.

[14] Koshiyama, A.S., Firoozye, N.B., & Treleaven, P.C. (2019). Generative adversarial networks for financial trading strategies fine-tuning and combination. Quantitative Finance, 21, 797 - 813.

[15] Wiese, M., Wood, B., Pachoud, A., Korn, R., Buehler, H., Murray, P., & Bai, L. (2021). Multi-Asset Spot and Option Market Simulation. ArXiv, abs/2112.06823.

[16] Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2019). Quant GANs: deep generation of financial time series. Quantitative Finance, 20, 1419 - 1440.

[17] Saxena, D., & Cao, J. (2020). Generative Adversarial Networks (GANs): Challenges, Solutions, and Future Directions. ArXiv, abs/2005.00065.

[18] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A.C. (2017). Improved Training of Wasserstein GANs. NIPS.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

[19] Lloyd, S., & Weedbrook, C. (2018). Quantum Generative Adversarial Learning. Physical review letters, 121 4, 040502 .

[20] Dallaire-Demers, P., & Killoran, N. (2018). Quantum generative adversarial networks. ArXiv, abs/1804.08641.

[21] Ganguly, S. (2021). Quantum Machine Learning: An Applied Approach: The Theory and Application of Quantum Machine Learning in Science and Industry. Quantum Machine Learning: An Applied Approach.

[22] Assouel, A., Jacquier, A., & Kondratyev, A. (2021). A quantum generative adversarial network for distributions. Quantum Machine Intelligence, 4, 1-19.

[23] Situ, H., He, Z., Wang, Y., Li, L., & Zheng, S. (2018). Quantum generative adversarial network for generating discrete distribution. Inf. Sci., 538, 193-208.

[24] Zhu, E.Y., Johri, S., Bacon, D., Esencan, M., Kim, J., Muir, M., Murgai, N., Nguyen, J., Pisenti, N.C., Schouela, A., Sosnova, K.E., & Wright, K. (2021). Generative quantum learning of joint probability distribution functions. Physical Review Research.

[25] Agliardi, G., & Prati, E. (2022). Optimal Tuning of Quantum Generative Adversarial Networks for Multivariate Distribution Loading. Quantum Reports.

[26] Zoufal, C., Lucchi, A., & Woerner, S. (2019). Quantum Generative Adversarial Networks for learning and loading random distributions. npj Quantum Information, 5, 1-9.

[27] Benedetti, M., Garcia-Pintos, D., Perdomo, O.M., Leyton-Ortega, V., Nam, Y.S., & Perdomo-Ortiz, A. (2018). A generative modeling approach for benchmarking and training shallow quantum circuits. npj Quantum Information, 5, 1-9.

[28] Cheng, S., Chen, J., & Wang, L. (2017). Information Perspective to Probabilistic Modeling: Boltzmann Machines versus Born Machines. Entropy, 20.

[29] Coyle, B., Mills, D., Danos, V., & Kashefi, E. (2019). The Born supremacy: quantum advantage and training of an Ising Born machine. npj Quantum Information, 6, 1-11.

[30] Arute, F., et al (2019). Quantum supremacy using a programmable superconducting processor. Nature, 574, 505 - 510.

[31] Liu, J., & Wang, L. (2018). Differentiable Learning of Quantum Circuit Born Machine. ArXiv, abs/1804.04168.

[32] Kondratyev, A., & Schwarz, C. (2019). The Market Generator. Econometrics: Econometric & Statistical Methods - Special Topics eJournal.

[33] Morini, M. (2009). One More Model Risk When Using Gaussian Copula for Risk Management. Risk Management eJournal.

[34] Zhu, D., Shen, W., Giani, A., Majumder, S.R., Neculaes, B., & Johri, S. (2022). Copula-based Risk Aggregation with Trapped Ion Quantum Computers.

[35] Chen, S.Y., Yoo, S., & Fang, Y.L. (2020). Quantum Long Short-Term Memory. *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 8622-8626.

[36] Foster, David. (2019). Generative Deep Learning - Teaching Machines to Paint, Write, Compose and Play.

[37] Lucas, A. (2013). Ising formulations of many NP problems. *ArXiv, abs/1302.5843*.

[38] Glover, F., Kochenberger, G. and Du, Y. (2019) A Tutorial on Formulating and Using QUBO Models. arXiv: 1811.11538. https://arxiv.org/abs/1811.11538 (last accessed 02nd of December, 2024)

[39] Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024, May 15). *Quantum computing with Qiskit*. arxiv.org/abs/2405.08810 (last accessed 02nd of December, 2024)

[40] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035).

[41] Bergholm, V., Izaac, J.A., Schuld, M., Gogolin, C., Khandelwal, A., & Killoran, N. (2018). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *ArXiv, abs/1811.04968*.

[42] Verma, A., & Lewis, M.W. (2021). Variable Reduction For Quadratic Unconstrained Binary Optimization. *ArXiv, abs/2105.07032*.(accessed on 22nd of November, 2024)

[43] August, M., & Ni, X. (2016). Using Recurrent Neural Networks to Optimize Dynamical Decoupling for Quantum Memory. *ArXiv, abs/1604.00279*.

[44] Flurin, E., Martin, L.S., Hacohen-Gourgy, S., & Siddiqi, I. (2018). Using a Recurrent Neural Network to Reconstruct Quantum Dynamics of a Superconducting Qubit from Physical Observations. *Physical Review X*.

[45] IBM. https://qiskit-community.github.io/

[46] Xanadu. https://pennylane.ai/qml/demos/tutorial_quanvolution/

[47] Bayraktar, H.H., Charara, A., Clark, D., Cohen, S., Costa, T.B., Fang, Y.L., Gao, Y., Guan, J., Gunnels, J.A., Haidar, A., Hehn, A., Hohnerbach, M., Jones, M.T., Lubowe, T., Lyakh, D.I., Morino, S., Springer, P.L., Stanwyck, S.W., Terentyev, I.S., Varadhan, S., Wong, J., & Yamaguchi, T. (2023). cuQuantum SDK: A High-Performance Library for Accelerating Quantum Science. *2023 IEEE International Conference on Quantum Computing and Engineering (QCE), 01*, 1050-1061.

[48] Xanadu. https://docs.pennylane.ai/projects/lightning/en/latest/lightning_gpu/device.html

[49] Ganguly, S. (2023). Implementing Quantum Generative Adversarial Network (qGAN) and QCBM in Finance. *ArXiv, abs/2308.08448*, accessed 15th December, 2024

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.

VIII. Appendix

A. QGAN Circuit

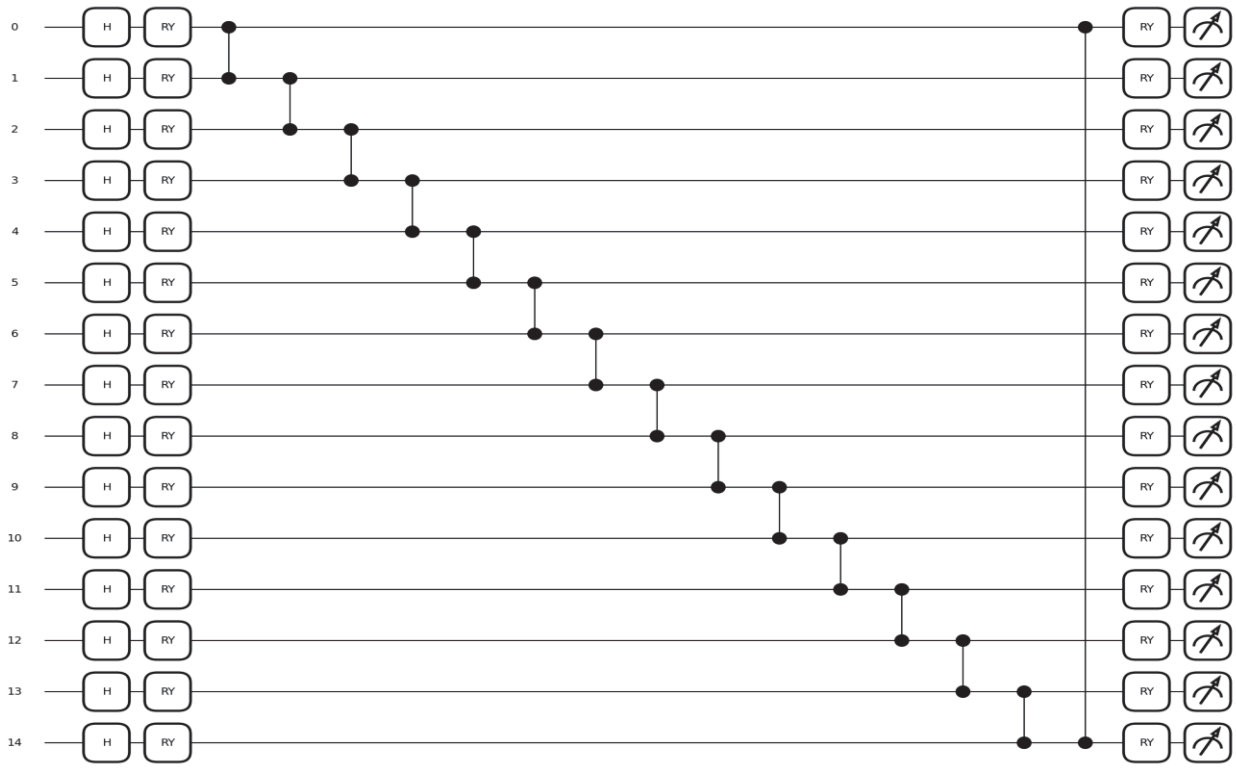

Fig A1. Quantum generator circuit for the qGAN

B. QLSTM

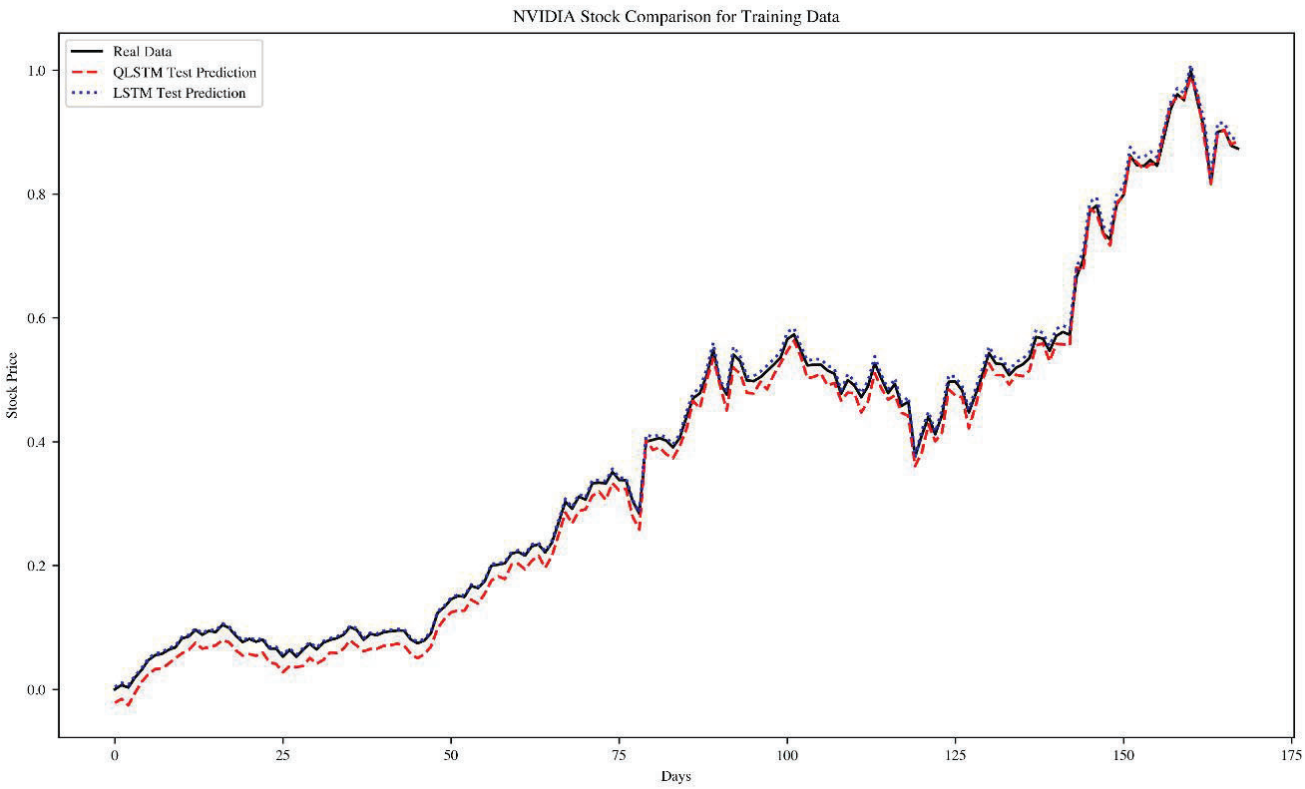

Fig A2. Comparison of Training between QLSTM, LSTM and Real data for the NVIDIA real life data. As the plot shows, training output was very similar.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:18:18 UTC from IEEE Xplore. Restrictions apply.